Cloud-Native Full Stack Application Using Spring Boot, React and Kubernetes.

Project Phase-I Report submitted to Osmania University, Hyderabad, in partial fulfillment of the requirement for the award of the degree of

Bachelor of Engineering

In

Information Technology

By

Mr. Mohd Ali	**(160923737137)**
Mr. Syed Ghyas Ahmed	**(160923737138)**
Mr. Mohammed Saif Ullah	**(160923737139)**

Under the Guidance of

Mrs. B Nagalakshhmi

Assistant Professor

DEPARTMENT OF INFORMATION TECHNOLOGY

LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY

(UGC Autonomous)

(Affiliated to Osmania University) Himayathsagar, Hyderabad,500091

A.Y. 2026-2027



LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY
(UGC Autonomous)
(Affiliated to Osmania University)
Himayathsagar, Hyderabad, 500091
Department of Information Technology

CERTIFICATE

This is to certify that the Project Phase-I report entitled **Cloud-Native Full Stack Application Using Spring Boot, React and Kubernetes.**being submitted by **Mohd Ali (160923737137), Syed Ghyas Ahmed (160923737138) and Mohammed Saif Ullah (160923737139)** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering** in **Information Technology** is a record of Bonafide work carried out by them.

Internal Project Guide
Mrs. B Nagalakshmi
Assistant Professor

Dr. K. Nagi Reddy
H.O.D, I.T

External Examiner



LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY
(UGC Autonomous)
(Affiliated to Osmania University)
Himayathsagar, Hyderabad, 500091
Department of Information Technology

DECLARATION BY THE CANDIDATES

We, **Mohd Ali (160923737137), Syed Ghyas Ahmed (160923737138) and Mohammed Saif Ullah (160923737139)**, hereby declare that the Project Phase-I report entitled **Cloud-Native Full Stack Application Using Spring Boot, React and Kubernetes.**is carried out under the guidance of **Mrs. B Nagalakshmi** Assistant Professor, Department of Information Technology, **Lords Institute of Engineering and Technology**, **Hyderabad** is submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering** in **Information Technology.**

This is a record of Bonafide work carried out by us and the results embodied in this project have not been reproduced or copied from any source. The results embodied in this project report have not been submitted to any other university or institute for the award of any other degree or diploma.

Mohd Ali (160923737137)
Syed Ghyas Ahmed (160923737138)
Mohammed Saif Ullah (160923737139)

ACKNOWLEDGMENT

We are very pleased to present this thesis of our project. This period of our student life has been truly rewarding a number of people were of immense help to us during the course of our research and the preparation of our thesis.

First, we wish to thank **GOD Almighty** who created heavens and earth, who helped us in completing this project and we also thank our parents who encouraged us in this period of research.

We would like to express our sincere gratitude to **The Management** of **Lords Institute of Engineering and Technology, Hyderabad** for providing us with an excellent platform and resources to successfully carry out this project.

We would also like to thank **Dr. Ravi Kishore Singh, Principal, L.I.E.T.** for extending his help in overall completion of this project work.

We would like to express my deep sense of gratitude **to Dr. K. Nagi Reddy, Professor & HOD**, **Information Technology, Lords Institute of Engineering & Technology, Hyderabad,** for his encouragement and cooperation throughout the project.

We would like to thank **Mrs. B Nagalakshmi, Assistant Professor, Dept. of Information Technology, Lords Institute of Engineering & Technology**, Hyderabad, our project internal guide, for her guidance and help. Her insight during the course of our research and regular guidance were invaluable to us.

Lastly, we acknowledge the support of our faculty members, friends, and family, who inspired and encouraged us throughout this journey.

Mohd Ali (160923737137)
Syed Ghyas Ahmed (160923737138)
Mohammed Saif Ullah (160923737139)

Vision of the Institute

Lords Institute of Engineering and Technology strives for excellence in professional education through quality, innovation and teamwork and aims to emerge as a premier institute in the state and across the nation.

Mission of the Institute

1. To impart quality professional education that meets the needs of present and emerging technological world.
2. To strive for student achievement and success, preparing them for life, career and leadership.
3. To provide a scholarly and vibrant learning environment that enables faculty, staff and students to achieve personal and professional growth.
4. To contribute to advancement of knowledge, in both fundamental and applied areas of engineering and technology.
5. To forge mutually beneficial relationships with government organizations, industries, society and the alumni.

PRINCIPAL
Principal
Lords Institute of Engineering & Tech.
(An Autonomous Institution)
Sy. No. 32, Himayath Sagar, Hyderabad-500091.T.S.

LORDS INSTITUTE OF ENGINEERING &TECHNOLOGY

Approved by AICTE/Affiliated to Osmania University/Estd.2003.
Accredited 'A' grade by NAAC
(UGC Autonomous Institution)

DEPARTMENT OF INFORMATION TECHNOLOGY

Vision of the Department:

To be a frontier in Information Technology focusing on quality professional education, innovation and employability.

Mission of the Department:

DM1: Adopt innovative teaching learning processes to achieve stated outcomes and sustained performance levels.

DM2: Provide learner centric environment to enrich knowledge, skills and innovation through industry academia collaborations.

DM3: Train the concerned stakeholders on latest technologies to meet the industry needs.

DM4: Inculcate leadership, professional, interpersonal skills and ethical values to address the contemporary societal issues.

Note: DM: Department Mission

Head of the Department
Head of the Department
Information Technology
Lords Institute of Engineering & Technology
Himayath Sagar, Hyderabad-91

B.E Information Technology Program Educational Objectives (PEOs):

PEO1	Be a competent software engineer in IT and allied industry providing viable solutions.
PEO2	Exhibit professionalism, engage in lifelong learning and adopt to changing industry and societal needs.
PEO3	Be effective as an individual and also cohesive in a group to execute multidisciplinary projects.
PEO4	Pursue higher studies in Information Technology with emphasis on competency and innovation.

Head of the Department
Head of the Department
Information Technology
Lords Institute of Engineering & Technology
Himayath Sagar, Hyderabad-91

B.E Information Technology Program Outcomes (POs):

Engineering Graduates will be able to:

S. No.	Program Outcomes (POs)
1.	**PO1: Engineering Knowledge:** Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
2.	**PO2: Problem Analysis:** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
3.	**PO3: Design/Development of Solutions:** Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
4.	**PO4: Conduct Investigations of Complex Problems:** Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
5.	**PO5: Engineering Tool Usage:** Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
6.	**PO6: The Engineer and The World:** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
7.	**PO7: Ethics:** Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
8.	**PO8: Individual and Collaborative Team work:** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
9.	**PO9: Communication:** Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences
10.	**PO10: Project Management and Finance:** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
11.	**PO11: Life-Long Learning:** Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Head of the Department
Head of the Department
Information Technology
Lords Institute of Engineering & Technology
Himayath Sagar, Hyderabad-91

B.E Information Technology Program Specific Outcomes (PSOs):

PSO1	**Professional Skills:** Identify, analyze requirements, design and implement computer programs in the area of artificial intelligence, machine learning, big data analytics and networking of varying complexity.
PSO2	**Problem-Solving Skills:** Deliver quality service to IT industry and provide solutions to real-life problems applying modern computer languages, software methods and tools.

Head of the Department
Head of the Department
Information Technology
Lords Institute of Engineering & Technology
Himayath Sagar, Hyderabad-91

Course Outcomes: Project Phase- I

After completion of the project, the student will be able to

S.No	Description	Blooms Taxonomy Level
PP.I.CO1	Demonstrate the ability to synthesize and apply the knowledge and skills acquired in the academic program to the real-world problems.	BTL3
PP.I.CO2	Evaluate different solutions based on economic and technical feasibility	BTL4
PP.I.CO3	Effectively plan a project and confidently perform all aspects of project management.	BTL5
PP.I.CO4	Demonstrate effective written and oral communication skills	BTL4
PP.I.CO5	Prepare the documentation report and perform the presentation of the project work	BTL3

LORDS INSTITUTE OF ENGINEERING &TECHNOLOGY
Approved by AICTE/Affiliated to Osmania University/Estd.2002.
Accredited 'A' grade by NAAC
(UGC Autonomous Institution)

DEPARTMENT OF INFORMATION TECHNOLOGY

Course Articulation Matrix:

Mapping of Course Outcomes (COs) with Program Outcomes (POs) and Program Specific Outcomes (PSOs):

Course Outcomes (COs)	Program Outcomes (POs)											Program Specific Outcomes (PSOs)	
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO 10	PO 11	PSO1	PSO2
PP.I.CO1	3	2	2	-	2	2	-	-	3	-	-	2	2
PP.I.CO2	3	3	2	3	2	-	2	-	3	-	-	-	2
PP.I.CO3	3	3	3	-	3	3	1	2	2	3	2	3	-
PP.I.CO4	3	2	2	3	3	3	3	2	-	-	2	3	3
PP.I.CO5	3	3	3	3	3	3	3	2	3	3	-	3	3
AVG	**3**	**2.6**	**2.5**	**3**	**2.6**	**2.7**	**2.2**	**2**	**2.9**	**3**	**2**	**2.7**	**2.5**

Level:

1. Low correlation (Low), 2- Medium correlation (Medium), 3-High correlation (High)

SDG Mapping:

SDG	Mapped Indicator	SDG	Mapped Indicator	SDG	Mapped Indicator
1 NO POVERTY		7 AFFORDABLE AND CLEAN ENERGY		13 CLIMATE ACTION	
2 ZERO HUNGER		8 DECENT WORK AND ECONOMIC GROWTH	✓	14 LIFE BELOW WATER	
3 GOOD HEALTH AND WELL-BEING		9 INDUSTRY, INNOVATION AND INFRASTRUCTURE	✓	15 LIFE ON LAND	
4 QUALITY EDUCATION	✓	10 REDUCED INEQUALITIES		16 PEACE, JUSTICE AND STRONG INSTITUTIONS	✓
5 GENDER EQUALITY		11 SUSTAINABLE CITIES AND COMMUNITIES		17 PARTNERSHIPS FOR THE GOALS	
6 CLEAN WATER AND SANITATION		12 RESPONSIBLE CONSUMPTION AND PRODUCTION			

ABSTRACT

The rapid growth of digital education has created a need for flexible, accessible, and scalable online learning platforms that allow students and instructors to interact through a unified digital environment. This project proposes the design and development of **Cloud-Native Full Stack Application Using Spring Boot, React and Kubernetes.**

The platform is designed to provide students with an efficient environment for discovering courses, enrolling in courses, accessing learning materials, completing quizzes, and tracking their learning progress.

The backend of the system is developed using Spring Boot, which provides RESTful APIs for user authentication, course management, enrollment, lesson management, quizzes, and progress tracking. The platform supports different user roles, including students, instructors, and administrators, with role-based access to their respective functionalities. A web-based frontend provides an interactive interface through which users can access the platform's services, while a relational database is used to securely store user, course, enrollment, assessment, and progress-related information.

To improve portability and deployment efficiency, the application components are packaged into Docker containers. Kubernetes is used for container orchestration, enabling efficient deployment, service management, scaling, and improved availability of the application. The system follows a modular architecture that allows its components to be independently managed and deployed.

The proposed platform demonstrates the integration of modern full-stack development with containerization and orchestration technologies to create a scalable and maintainable online learning environment. The project provides practical implementation of Spring Boot, RESTful APIs, database management, authentication, Docker, Kubernetes, and cloud-oriented deployment, making the system suitable for modern educational applications.

Keywords: Cloud-native application, Spring Boot, React, Kubernetes, Docker, REST API, microservices principles, container orchestration, CI/CD, observability.

CONTENTS

Certificate i

Declaration ii

Acknowledgement iii

Vision and Mission of the Institute iv

Vision and Mission of the Department v

Program Educational Objectives vi

Program Outcomes vii

Program Specific Outcomes viii

Course Outcomes ix

Course Articulation Matrix x

Abstract xi

Contents xii

List of Figures xv

List of Tables xvi

CHAPTER-1: INTRODUCTION 1

1.1 Introduction 1

1.2 Problem Statement 1

1.3 Objectives and Scope 2

1.4 Existing System 2

1.5 Limitations of Existing System 3

1.6 Proposed System 3

1.7 Advantages 3

CHAPTER-2: LITERATURE SURVEY 4

2.1 Introduction 4

2.2 Cloud-Native Full Stack Application Using Spring Boot 4

2.3 Secure Application Management Using Cloud Services and ReactJS 5

2.4 Containerization and Orchestration 5

2.5 Comparison of Related Approaches 6

2.6 Research Gap and Proposed Contribution 7

2.7 Summary 7

CHAPTER-3: SYSTEM ANALYSIS AND REQUIREMENTS 8

3.1 Introduction 8

3.2 Functional Requirements 8

3.3 Non-Functional Requirements 9

3.4 Users Roles and Use Cases 10

3.5 Assumptions and Constrains 11

CHAPTER-4: SYSTEM DESIGN AND ARCHITECTURE 12

4.1 Introduction 12

4.2 Feasibility Study 12

- 4.2.1 Technical Feasibility 12
- 4.2.2 Economic Feasibility 12
- 4.2.3 Operational Feasibility 13
- 4.2.4 schedule Feasibility 13

4.3 Architecture and Components Design 14

- 4.3.1 Request processing Flow 15
- 4.3.2 Design Principles 16

4.4 Data and API Design 16

- 4.4.1 Error Response Design 17

4.5 Deployment Design 17

4.6 Deployment Workflow 19

CHAPTER-5: IMPLEMENTATION AND TESTING 20

5.1 Introduction 20

5.2 Input Design 20

- 5.2.1 Main List and Filter Inputs 20
- 5.2.2 Create and Update Form 21
- 5.2.3 Input Security 22

5.3 Output Design 22

- 5.3.1 List View Design 23
- 5.3.2 Detail and Confirmation Output 23

5.4 Validation and Error Handling 24

5.5 Acceptance Criteria 25

CHAPTER-6: RESULTS AND FUTURE ENHANCEMENTS **26**

6.1 Introduction 26

6.2 Software Requirements 26

6.3 Development and Deployment Tools 27

- 6.3.1 Development Environment 27
- 6.3.2 Docker Environment 27
- 6.3.3 Kubernetes Environment 28

6.4 Security and Observability 28

- 6.4.1 Security Controls 28
- 6.4.2 Logging, Metrics and Health 28

6.5 Implementation Plan 29

6.6 Future Enhancements 29

CONCLUSION **30**

REFERENCES **31**

List of Figures

SI.no	Fig.no	Name of the Figure	Page No
1	4.1	Proposed Architecture of the Cloud-Native Full Stack Application	14
2	4.3	Deployment Architecture of the Cloud-Native Application	18

List of Tables

SI.no	Table.no	Name of the Table	Page No
1	2.1	Comparison	6
2	3.1	Functional Requirements	8
3	3.2	Non-Functional Requirements	9
4	4.1	Feasibility	13
5	4.2	Architecture	15
6	4.3	Data and API	16
7	4.4	Deployment Design	18
8	5.1	Lists and Inputs	21
9	5.2	Create and Update	22
10	5.3	API Output Contract	23
11	5.4	Testing Strategy	24
12	6.1	Software Requirement	26

CHAPTER-1
INTRODUCTION

1.1 Introduction

Cloud-native development is an approach to building and operating applications that are designed to take advantage of automation, containers, declarative infrastructure and distributed runtime platforms. Instead of treating deployment as a final manual step, cloud-native engineering considers packaging, configuration, monitoring, scaling and recovery as part of the application lifecycle. These practices are useful for systems that evolve frequently or need to serve users from different environments.

A full stack application includes the presentation layer, server-side business logic and data persistence. In this project, React is responsible for the presentation layer and user interactions, while Spring Boot provides the backend REST API. The backend communicates with a database through a persistence layer. Docker packages the application into portable images, and Kubernetes provides the runtime orchestration layer. This separation makes it easier to develop, test and deploy the frontend and backend independently.

The project is designed as a practical reference implementation rather than a single fixed business domain. The core platform can support records such as projects, products, tasks, requests or learning resources. Domain-specific fields and workflows can be added without changing the overall deployment model. The architecture aims to demonstrate clear separation of concerns, controlled configuration, repeatable builds and reliable release practices.

1.2 Problem Statement

Applications developed and deployed without a consistent delivery strategy can experience environment differences, slow releases, limited scalability and difficult recovery after failures. A frontend tightly coupled to backend implementation details can be hard to maintain. Manual server configuration can introduce inconsistencies, while deployments that lack health checks and rollout controls can cause avoidable downtime. Logs scattered across components also make it harder to diagnose problems.

The problem addressed by this project is how to design a maintainable full stack application that can be packaged and deployed consistently, expose clear APIs, validate user input,

manage configuration safely and use orchestration capabilities to improve operational repeatability. The system should be understandable to developers and suitable for extension into a larger production application.

1.3 Objectives and Scope

Build a responsive user interface using React components and client-side routing.

Develop REST APIs using Spring Boot with clear controller, service and repository responsibilities.

Provide validation, meaningful error responses and consistent API contracts.

Persist application records in a relational database through a defined data-access layer.

Package the frontend and backend as container images using Docker.

Deploy workloads to Kubernetes with Deployments, Services, configuration resources and health probes.

Establish a foundation for automated build, test and deployment pipelines.

Document security, monitoring, testing and future improvement considerations.

The scope includes the core architecture, application workflow, API and data design, container packaging and Kubernetes deployment design. It does not assume a specific cloud provider, production traffic level or completed third-party compliance assessment. Those details must be selected and verified for the intended organization.

1.4 Existing System

In a basic application environment, the frontend, backend and database may be deployed through separate manual steps. Developers may configure machines indivisually, copy build artifacts to servers and update configuration files by hand. This approach can be sufficient for a small prototype, but it becomes difficult to repeat consistently across development, testing and production environments.

In some existing systems, user-interface logic is tightly coupled to server-rendered pages or backend-specific response structures. Filtering and sorting may be implemented only in the browser, which can become inefficient for large datasets. Error responses may vary between endpoints, and the frontend may not receive enough information to explain a failure to the user. Operational checks may depend on a person manually inspecting logs or restarting a process.

1.5Limitations of the Existing System

- Manual deployment can lead to differences between environments and increase release time.
- Scaling may require manual server provisioning and application configuration.
- Application failures may not be detected promptly if health checks are absent.
- Frontend and backend changes can become difficult to coordinate without stable API contracts.
- Configuration values may be duplicated or accidentally committed to source control.
- Insufficient centralized logging can slow down root-cause analysis.
- Large list responses can waste bandwidth if pagination and server-side filtering are not provided.
- Testing may be inconsistent if build, test and deployment steps are not automated.

1.6 Proposed System

The proposed system separates the React frontend, Spring Boot backend and data store. The frontend sends HTTP requests to the backend API. The backend validates the request, applies business rules and accesses the database through a repository layer. Docker images package the frontend and backend, while Kubernetes manages the running workloads. Configuration is supplied externally so that the same image can be promoted across environments.

1.7 Advantages

Improved User Experience: Enables users to share their feedback easily.
Better Decision-Making: Helps users make informed choices based on ratings and reviews.
Increased Transparency: Displays average ratings and total review counts.
Data Validation: Ensures ratings and reviews follow predefined validation rules.
Secure Feedback: Protects users' personal information and maintains data privacy.
Higher User Engagement: Encourages users to actively participate by sharing their experiences.

CHAPTER-2
LITERATURE SURVEY

2.1 Introduction

The literature survey provides an overview of existing systems, technologies, and research related to the proposed application. It helps identify the methods, tools, and approaches used in previous systems. This study also highlights existing limitations and provides a foundation for developing an improved and efficient system.

2.2 Cloud-Native Full Stack Application Using Spring Boot

Modern web application design commonly separates the presentation layer from server-side business logic through an API boundary. A browser-based frontend can evolve independently when it communicates with backend services through a stable contract. React supports component reuse and state-driven rendering, while Spring Boot reduces the configuration needed to build Java-based web services. Together, these technologies provide a practical approach for developing maintainable business applications.

A layered backend design normally separates controllers, services and repositories. Controllers handle HTTP concerns and translate requests into application operations. Services enforce business rules and coordinate work. Repositories isolate persistence details. This separation improves testability because service behaviour can be tested independently of the web layer and database implementation. DTOs can be used to avoid exposing persistence entities directly as public API contracts.

The cloud-native aspect extends beyond choosing a cloud hosting provider. It includes immutable application images, externalized configuration, automated deployment, health checks and observable runtime behaviour. Containerization creates a repeatable unit of deployment, while an orchestrator schedules workloads and maintains the declared desired state. These practices reduce dependence on manual server setup, although they introduce additional operational concepts that teams must understand.

2.3 Secure Application Management Using Cloud Services and ReactJS

A React-based interface can provide responsive forms, tables, filtering, sorting and status indicators. However, client-side validation is only a usability measure and cannot be treated as a security boundary. All security-sensitive rules must be enforced by the backend. Requests must be authenticated where required, authorization must be checked for each protected operation, and sensitive configuration such as credentials must not be embedded in frontend bundles.

Cloud-hosted systems commonly use managed databases, container registries, secrets services, identity providers and monitoring platforms. Managed services can reduce operational burden, but they require careful configuration of access permissions, network exposure, backup policies and cost controls. The application should use the minimum permissions needed and avoid exposing administrative interfaces or database ports to the public internet.

A secure design also considers transport encryption, dependency updates, request limits, audit records and safe error messages. Logs should provide enough context to investigate failures without recording passwords, access tokens or unnecessary personal information. Security testing should cover authentication, authorization, input validation, cross-origin rules and common web application risks.

2.4 Containerization and Orchestration

Docker images package application code with the runtime dependencies required to execute it. A well-designed image uses a predictable base image, separates build-time and runtime stages when practical, avoids running as a privileged user and excludes unnecessary files. Image tags should identify a build or release rather than relying only on a mutable latest tag. Vulnerability scanning and image provenance checks can be incorporated into the delivery pipeline.

Kubernetes provides declarative resources for managing containerized workloads. A Deployment describes the desired number of application replicas and the image to run. A Service provides a stable network identity for a group of Pods. ConfigMaps hold non-sensitive configuration, while Secrets are intended for sensitive values and should be protected through appropriate cluster configuration and access control. Readiness probes determine when a workload can receive traffic; liveness probes help detect a process that needs restarting.

Orchestration is not a substitute for application design. Applications still need sensible timeouts, connection pool settings, graceful shutdown, retry policies and database migration practices. Multiple replicas may improve availability, but they do not automatically make the database highly available or remove every single point of failure. The whole dependency chain must be considered.

2.5 Comparison of Related Approaches

Table 2.1 Comparison

Approach	Strengths	Limitations / Considerations
Single-unit application deployed manually	Simple to start; low initial operational complexity	Scaling and release coordination become difficult as the application grows
Frontend and backend separated by REST API	Clear responsibilities; independent development and testing	Requires API versioning, CORS configuration and coordinated contract changes
Docker-based deployment	Portable packaging and repeatable runtime	Image security, tagging and storage must be managed
Kubernetes-based deployment	Declarative operations, service discovery, rollout and scaling controls	Adds cluster, networking, resource and security complexity
Proposed combined approach	Connects user experience, API design, packaging and orchestration in one workflow	Needs automated testing, monitoring and operational ownership to be effective
Approach	Strengths	Limitations / Considerations
Single-unit application deployed manually	Simple to start; low initial operational complexity	Scaling and release coordination become difficult as the application grows
Frontend and backend separated by REST API	Clear responsibilities; independent development and testing	Requires API versioning, CORS configuration and coordinated contract changes
Docker-based deployment	Portable packaging and repeatable runtime	Image security, tagging and storage must be managed

2.6 Research Gap and Proposed Contribution

The technologies used in this project are established technologies rather than a new algorithm. The project contribution is the integration of frontend development, backend API design, container packaging and orchestration into a coherent, documented application lifecycle. It emphasizes practical implementation decisions that are often considered separately: validation, error handling, configuration, health checks, release strategy and observability.

The proposed solution uses a modular architecture with clear interfaces and a deployment model that can be adapted to a range of business domains. Evaluation should focus on functional correctness, deployment repeatability, API response behavior, recovery from a controlled failure and the effort required to release a new version. Any claims about improved performance or availability should be supported by measurements collected during testing.

2.7 Summary

The survey indicates that a reliable cloud-native application requires more than a frontend framework and a backend framework. API contracts, secure configuration, automated packaging, orchestration and monitoring must work together. These observations guide the requirements and design described in the following chapters.

CHAPTER-3
SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Introduction

System analysis identifies the functional capabilities, technical requirements, user roles, and operational constraints of the proposed platform.

The requirements are divided into functional requirements and non-functional requirements to guide implementation and testing.

3.2 Functional Requirements

Table 3.1 Functional Requirements

ID	Requirement	Expected Behavior
FR-01	View records	Users can open the main list and view records they are permitted to access.
FR-02	Create record	Authorized users can submit a valid form to create a record.
FR-03	Update record	Authorized users can update permitted fields of an existing record.
FR-04	Delete or deactivate	The system applies the configured removal policy and confirms the result.
FR-05	Filter and search	Users can filter by supported fields and search by keyword.
FR-06	Sort and paginate	The API returns ordered pages with metadata and bounded page size.

FR-07	Validation	Invalid requests receive actionable field or request-level error messages.
FR-08	Ratings and reviews	If enabled, users can submit valid ratings and reviews under defined rules.
FR-09	Health endpoints	The runtime exposes health information suitable for deployment probes.
FR-10	Audit / operational logs	Important actions and failures are recorded without sensitive secrets.

3.3 Non-Functional Requirements

Table 3.2 Non-Functional Requirements

Category	**Requirement**
Performance	Common requests should meet an agreed response-time target under a defined test load.
Availability	The deployment should use health checks and controlled rollouts; availability depends on the full infrastructure.
Security	Use authenticated access where needed, role checks, input validation, secure transport and protected secrets.
Maintainability	Use modular frontend components, layered backend code, tests and documented API contracts.
Scalability	Support increasing workload through bounded resources, pagination and replica

	scaling where appropriate.
Portability	Package applications as container images and externalize environment-specific configuration.
Usability	Forms, loading states, empty states and errors should be clear and accessible.
Observability	Provide logs, health signals and metrics sufficient to identify failures and resource pressure.
Recoverability	Document backup, restore, rollback and recovery procedures for the selected environment.

3.4 User Roles and Use Cases

The exact roles depend on the business domain, but a typical system may define an administrator, a standard user and a read-only viewer. Administrators manage configuration and users; standard users create or update records according to permissions; viewers can inspect information without changing it. The backend must enforce these permissions even if the interface hides controls from unauthorized users.

1. User opens the application and loads the main dashboard.
2. User requests a list of records, optionally supplying search, filter, sort and pagination parameters.
3. Backend validates query parameters, retrieves permitted records and returns a structured response.
4. User opens a record form and submits data.
5. Backend validates the payload, applies business rules, persists the change and returns a success response.
6. Frontend updates the view and displays a clear confirmation or actionable error.

3.5 Assumptions and Constraints

The project assumes access to a supported Java Development Kit, Node.js runtime, container tooling and a Kubernetes environment for deployment testing. The selected database must be reachable from the backend and configured through external settings. A production deployment requires environment-specific decisions about ingress, TLS, identity, backups, monitoring, resource limits and cloud cost. The report does not claim that these operational controls are automatically supplied by Kubernetes.

CHAPTER-4
SYSTEM STUDY AND DESIGN

4.1 Introduction

This chapter explains the overall design and architecture of the proposed cloud-native full stack application. It describes the interaction between the React frontend, Spring Boot backend and database. It also covers the role of Docker and Kubernetes in containerization and deployment. The chapter discusses the system architecture, component design, data design and API design.

4.2 Feasibility Study

4.2.1 Technical Feasibility

The proposed stack is technically feasible because React, Spring Boot, Docker and Kubernetes are widely used technologies with established documentation and tooling. React supports component-based user interfaces; Spring Boot supports REST services and integration with persistence libraries; Docker provides image-based packaging; and Kubernetes manages container workloads. A developer can implement and test the application locally before deploying it to a cluster.

Technical feasibility depends on the team's familiarity with API design, container networking, Kubernetes resources and secure configuration. A small proof of concept should verify database connectivity, image build, service routing, health probes and configuration injection before full feature development. For a small academic project, a local Kubernetes distribution or managed cluster can be used, provided its limitations are documented.

4.2.2 Economic Feasibility

The core frameworks and development tools are available without per-developer licensing fees, although hosting, managed databases, registries, monitoring and network traffic may incur costs. Local development can reduce initial infrastructure expense. Cloud resources should be sized conservatively, stopped or removed when not required, and monitored for unexpected usage. The economic assessment should compare the expected value of repeatable deployment and maintainability with the cost of operating the selected platform.

4.2.3 Operational Feasibility

The application is operationally feasible when users can complete common tasks through a clear interface and administrators can deploy, monitor and troubleshoot it using documented procedures. Operational readiness includes a deployment runbook, environment configuration, access management, database backup and restore steps, and a defined rollback procedure. Kubernetes reduces some manual work but does not remove the need for operational ownership.

4.2.4 Schedule Feasibility

Table 4.1 Feasibility

Phase	Activities	Indicative Duration
1	Requirements, use cases and interface sketches	1 week
2	Backend model, REST APIs and persistence	2 weeks
3	React screens, routing and API integration	2 weeks
4	Unit and integration testing; validation and error handling	1 week
5	Docker images and local container testing	1 week
6	Kubernetes manifests, probes and deployment verification	1–2 weeks
7	Documentation, demonstration and final corrections	1 week

The schedule is indicative and depends on the number of developers, domain complexity, infrastructure availability and the depth of testing required. Security review and production readiness may require additional time.

4.3 Architecture and Component Design

The system uses a layered architecture with a separately deployed frontend and backend. The browser loads the React application from a web server or static hosting service. API calls are sent to the Spring Boot service through a defined base URL. The backend applies authentication and authorization where configured, validates request data, invokes business services and accesses the database. The backend returns JSON responses that the frontend renders as tables, forms, notifications or detail views.

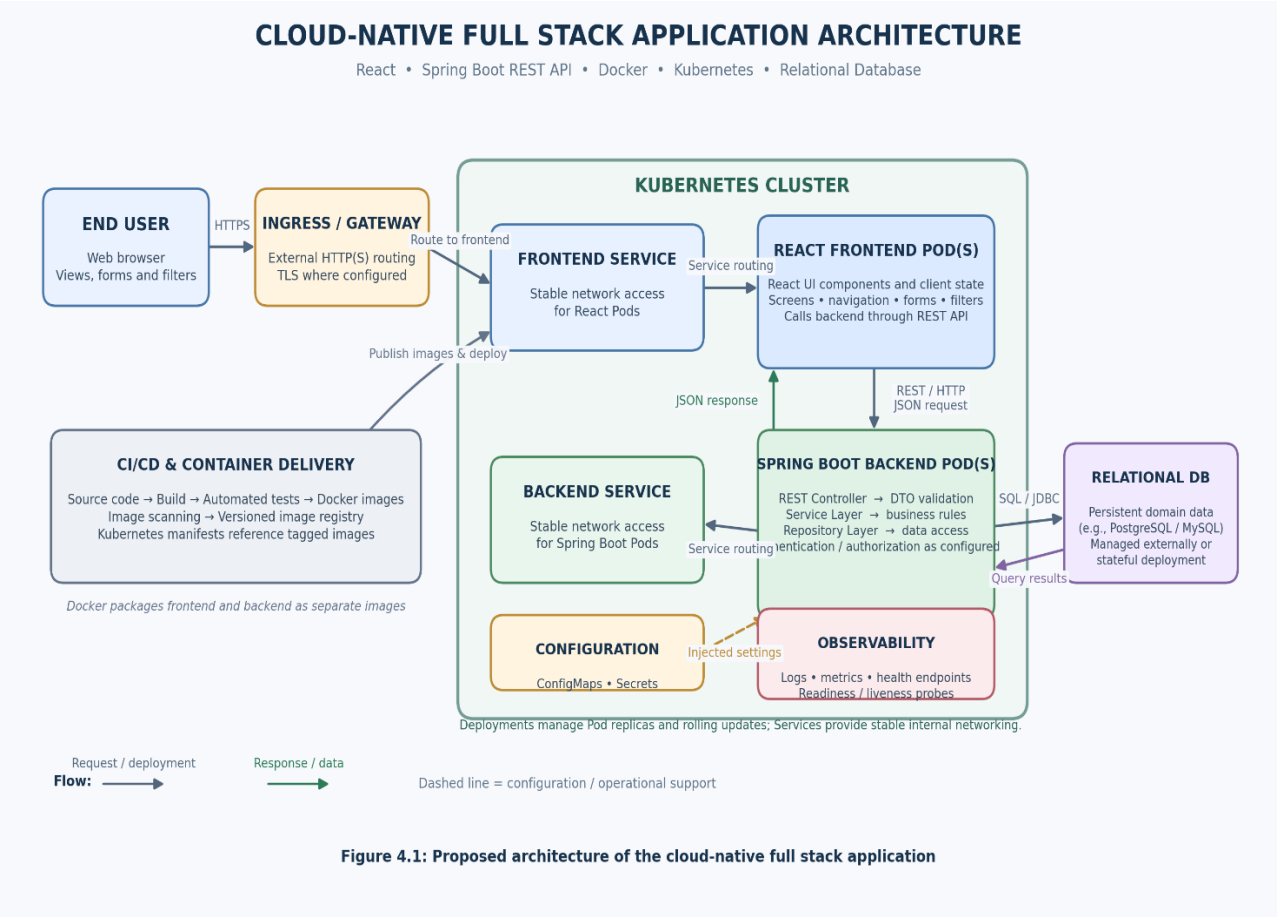


Figure 4.1: Proposed architecture of the cloud-native full stack application

Table 4.2 Architecture

Component	Responsibility	Design Notes
React UI	Screens, forms, navigation, filters and client-side state	Keep API access in a service module; show loading, empty and error states.
REST Controller	HTTP routing and request/response mapping	Use DTOs, correct status codes and consistent error structures.
Service Layer	Business rules and transaction coordination	Keep domain decisions out of controllers.
Repository Layer	Database access	Use parameterized queries and appropriate indexes.
Relational Database	Durable storage for domain records	Define keys, constraints, migrations and backup strategy.
Docker	Build and package application images	Use repeatable builds, minimal runtime images and non-root execution.
Kubernetes	Schedule and expose workloads	Use Deployments, Services, probes, resource requests and limits.
Observability	Logs, metrics and health signals	Avoid logging credentials or unnecessary personal information.

4.3.1 Request Processing Flow

1. The user performs an action in the React interface.
2. The frontend checks basic field constraints and sends an HTTP request to the API.
3. The controller maps the request into a validated DTO.
4. The service layer checks business rules and authorization context.
5. The repository reads or writes data within the configured transaction boundary.
6. The backend returns a success response or a consistent error response.
7. The React interface updates the displayed data and provides feedback.

4.3.2 Design Principles

- Separation of concerns: each layer has a defined responsibility.
- Stateless API requests where practical, enabling horizontal scaling of backend replicas.
- Externalized configuration for URLs, credentials and environment-specific settings.
- Defensive input validation and consistent error handling.
- Least privilege for application identities, database access and cluster permissions.
- Repeatable builds and versioned container images.
- Health checks and resource limits configured according to observed behaviour.

4.4 Data and API Design

A domain record should have a stable identifier, required business fields, timestamps where useful and a clearly defined lifecycle. The final entity model depends on the selected application domain. For example, a project-tracking application may contain Project, User, Project Status and Review entities. A catalogue application may contain Item, Category, Rating and Review entities. Relationships should be represented with explicit foreign keys and appropriate uniqueness constraints.

Table 4.3 Data and API

Example Resource	Endpoint	Purpose
Records	GET /api/v1/records	List records with filters, sort and pagination.
Record	GET /api/v1/records/{id}	Retrieve a record by identifier.
Record	POST /api/v1/records	Create a record after validation.
Record	PUT /api/v1/records/{id}	Replace or update a record according to API rules.
Record	DELETE /api/v1/records/{id}	Delete or deactivate a record according to policy.
Health	GET /actuator/health	Provide application health information when configured.

Reviews	POST /api/v1/records/{id}/reviews	Optional endpoint to submit a rating and review.

These routes are illustrative API contracts. The final implementation must define exact request DTOs, response DTOs, status codes, authentication requirements and error schemas. API versioning helps manage future contract changes. Pagination responses should include current page, page size, total elements or continuation information as appropriate.

4.4.1 Error Response Design

Errors should be consistent and safe. A response can include a timestamp, HTTP status, stable error code, short message, request path and field-level validation details. It should not expose stack traces, SQL statements, internal hostnames or secrets. The frontend should translate common errors into understandable messages and should preserve form values when a submission fails.

Large files should preferably be stored in object storage or a dedicated media service rather than directly in relational database records.

4.5 Deployment Design

The frontend and backend are built into separate images. A continuous integration pipeline can install dependencies, run tests, build artifacts, create images, scan dependencies and publish versioned images to a registry. Deployment manifests reference explicit image tags. The cluster pulls the images and runs them as Pods under Deployments. Services provide stable internal access, and an Ingress or gateway can route external traffic when required.

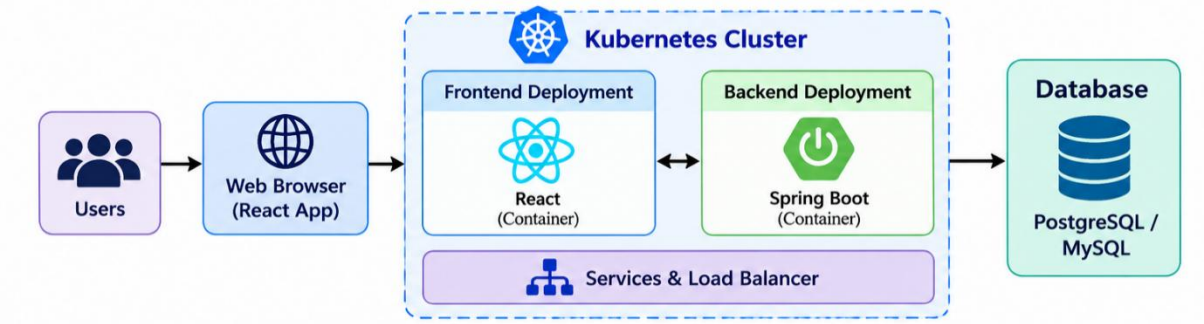


Figure 4.3: Deployment Architecture of the Cloud-Native Application

Table 4.3: Deployment Components and Description

Component	Description
Docker Containers	Packages the frontend and backend applications for consistent deployment.
Kubernetes Cluster	Manages container orchestration, scaling and self-healing.
Services & Load Balancer	Exposes the application and distributes traffic.
Database	Stores application data securely.
Monitoring Tools	Tracks performance and health of the application.
Secrets & ConfigMaps	Manages sensitive data and configuration settings.

Table 4.4 Deployment Design

Kubernetes Resource	**Use in the Project**
Deployment	Declares replica count, image version and Pod template.
Service	Provides stable networking for frontend or backend Pods.
ConfigMap	Supplies non-sensitive environment configuration.
Secret	Supplies sensitive values, subject to cluster encryption and access controls.
Readiness Probe	Prevents traffic being sent to a Pod before it is ready.
Liveness Probe	Detects an unhealthy process that may need restarting.
Resource Requests/Limits	Helps scheduling and bounds resource consumption.
Ingress / Gateway	Provides external HTTP routing and TLS integration where configured.

The database may be managed externally or deployed using an approved stateful design. Persistent data must not rely on a container's ephemeral filesystem. Database credentials should be injected securely, and schema changes should be applied through a controlled migration process. Before production release, the team should test rolling updates, rollback, Pod restart, loss of a dependency and database recovery.

4.6 Deployment Workflow

1. The React frontend and Spring Boot backend are developed and tested separately.

2. Docker is used to build container images for both application components.

3. The container images are made available to the deployment environment.

4. Kubernetes Deployments manage the application Pods, while Services provide stable communication between components.

5. ConfigMaps and Secrets provide application configuration and sensitive information.

6. Health probes help Kubernetes identify application readiness and failures.

7. Monitoring and logging help track application performance and identify problems.

CHAPTER-5

INPUT AND OUTPUT DESIGN

5.1 Introduction

This chapter describes the implementation of the cloud-native full stack application using React and Spring Boot. It explains how the frontend and backend components are developed and integrated. It also discusses the use of Docker for creating container images and Kubernetes for managing application deployment. The chapter focuses on the practical development of the proposed system.

5.2 Input Design

Input design defines how users provide data and how the system checks that data before processing. Forms should use meaningful labels, identify mandatory fields, provide examples when a format may be unclear and display validation feedback near the relevant field. Controls should be keyboard accessible, and error messages should not rely on color alone. A user should be able to understand what is required without consulting technical documentation.

5.2.1 Main List and Filter Inputs

The list screen may provide a keyword search field, status selector, category selector, date range, sort selector and pagination controls. The UI should clearly indicate active filters and provide a reset action. Search should have a defined matching policy, such as case-insensitive substring matching for selected fields. The API should validate filter names and sort fields instead of accepting arbitrary database properties.

Table 5.1 Lists and Inputs

Input Field	Type / Example	Validation / Behaviour
Keyword	Text, up to an agreed length	Trim whitespace; apply safe server-side search rules.
Status	Dropdown	Accept only known status values.
Category	Dropdown or multi-select	Verify selected values against allowed categories.
Start date	Date	Must be a valid date and not later than end date.
End date	Date	Must be valid and compatible with start date.
Page	Integer	Must be zero or greater, according to API convention.
Page size	Integer	Apply a configured maximum.
Sort	Allowed field and direction	Reject unsupported fields and directions.

5.2.2 Create and Update Form

A create or update form should present only fields relevant to the selected domain. Required fields should be marked clearly, and the interface should prevent accidental double submission while a request is in progress. The backend remains authoritative for validation because requests can be sent without using the browser interface. After a successful submission, the UI can navigate to the detail view or return to the refreshed list.

Table 5.2 Create and Update

Example Field	Input Rule	Purpose
Title / Name	Required; bounded length	Identifies the record.
Description	Optional or required according to domain; bounded length	Explains the record.
Category	Must match an allowed category	Supports grouping and filtering.
Status	Must be a permitted state transition	Tracks the lifecycle.
Date	Valid date within business constraints	Supports planning and filtering.
Rating	Integer or decimal within the selected scale	Optional feedback feature.
Review text	Bounded length; moderation rules may apply	Optional written feedback.

5.2.3 Input Security

All inputs should be treated as untrusted. The server should validate type, length, allowed values and business constraints. Database access must use parameterized operations. Output encoding should be applied in the appropriate rendering context, and file upload functionality, if added, must use size limits, content checks and safe storage. Authentication and authorization checks must be applied to protected operations.

5.3 Output Design

Output design covers the information returned by the backend and the way it is presented to the user. Outputs should be accurate, consistent and appropriate for the user's role. List screens should display useful columns without overwhelming the user. Detail views should group related fields, and confirmation messages should state what action succeeded. If a request fails, the interface should explain the next step where possible.

5.3.1 List View Output

Display record title or name, status, category and relevant timestamps.

Show a clear empty state when no records match the current filters.

Include pagination controls and the current page context for large result sets.

Show active filter chips or a concise summary of applied filters.

Provide actions only when the current user has permission to perform them.

Use a consistent date, number and status format throughout the application.

5.3.2 Detail and Confirmation Output

A detail page should show the selected record's key attributes, status and related information. If ratings and reviews are enabled, the screen can display the average rating, number of reviews and individual review entries, subject to privacy and moderation rules. After a create, update or delete operation, the interface should show a confirmation and refresh any affected data. Destructive actions should require confirmation where accidental activation could cause data loss.

Table 5.3 API Output Contract

Response Type	Typical HTTP Status	Expected Output
Successful list	200 OK	Content collection plus pagination metadata.
Successful detail	200 OK	Requested record representation.
Created record	201 Created	Created representation and/or resource location.
Successful update	200 OK or 204 No Content	Updated representation or empty success response.
Successful delete	204 No Content or documented alternative	No content or a consistent operation result.
Invalid input	400 Bad Request	Validation errors with safe field-level details.
Unauthenticated	401 Unauthorized	Generic authentication-required response.
Forbidden	403 Forbidden	Indicates the operation is not permitted.

Not found	404 Not Found	Safe message for a missing resource.
Unexpected failure	500 Internal Server Error	Generic message plus a correlation identifier if available.

5.4 Validation and Error Handling

Validation should occur at the UI boundary for fast feedback and again at the API boundary for correctness. The service layer should enforce business rules that involve existing data or user permissions. Database constraints provide a final integrity boundary for uniqueness and relationships. Error messages should be specific enough to help users correct input but should not disclose implementation details.

The frontend should distinguish validation errors from network failures, authentication problems and server errors. A loading indicator should be shown during longer requests, and repeated submissions should be prevented where possible. If a request times out, the UI should avoid assuming that the server did not process it; for operations that are not naturally idempotent, the design may need a safe retry strategy.

Table 5.4 Testing Strategy

Test Level	**Focus**	**Example**
Unit testing	Small units of logic	Test service validation and state-transition rules.
Controller/API testing	HTTP contract and status codes	Test invalid payloads and missing resources.
Repository/integration testing	Persistence behaviour	Verify constraints, queries and transactions.
Frontend component testing	Rendering and user interaction	Test filter changes, validation and empty states.
End-to-end testing	Complete user journeys	Create a record, filter it and update its status.
Container testing	Image startup and configuration	Run images with environment-specific configuration.

Kubernetes deployment testing	Scheduling, probes and routing	Verify readiness, restart and rolling update behaviour.
Security testing	Access control and unsafe inputs	Test forbidden operations and secret handling.
Load testing	Behaviour under agreed workload	Measure latency, errors and resource consumption.

5.5 Acceptance Criteria

- Users can complete the main supported workflows without unhandled UI errors.
- Required fields and invalid values are rejected with clear feedback.
- Filters and pagination return results consistent with documented API behaviour.
- Unauthorized users cannot perform protected operations through direct API calls.
- Container images start successfully using external configuration.
- Kubernetes readiness and liveness checks behave as documented.
- A new application version can be deployed and rolled back using the documented process.
- Logs and health information support basic troubleshooting without exposing secrets.

CHAPTER-6
RESULTS AND FUTURE ENHANCEMENTS

6.1 Introduction

This chapter describes the software environment and tools required for developing and deploying the cloud-native full stack application. It explains the software requirements, development tools and deployment technologies used in the project. It also discusses the security and observability aspects of the application. Finally, it presents the conclusion and future enhancements of the proposed system.

6.2 Software Requirements

Table 6.1 Software Requirements

Software / Platform	**Purpose**	**Notes**
Java Development Kit	Compile and run the Spring Boot backend	Use a Java version supported by the selected Spring Boot release.
Spring Boot	REST API and application framework	Choose compatible dependency versions and maintain them.
Maven or Gradle	Backend build and dependency management	Use a reproducible build configuration.
Node.js and npm	React development and frontend build	Use a supported LTS release for project stability.
React	Frontend component library	Use a consistent component and state-management approach.
Database	Persistent application data	Examples include PostgreSQL or MySQL; select one for implementation.
Docker	Build and run container images	Use multi-stage builds where appropriate.

Kubernetes	Container orchestration	Can be local for development or managed for deployment.
Git	Version control	Use branches, reviews and tagged releases.
API client	Manual REST API verification	Examples include Postman, Insomnia or curl.
Testing framework	Automated test execution	JUnit and Mockito for Java; a suitable React test framework for UI.
CI/CD platform	Automated build, test and release	Select according to the hosting environment.

6.3 Development and Deployment Tools

6.3.1 Development Environment

Development can be performed using an IDE such as IntelliJ IDEA, Eclipse or Visual Studio Code. The repository should separate frontend and backend source code clearly or use a documented multi-module layout. Environment files containing local secrets must be excluded from version control. Example configuration files should contain placeholders rather than usable credentials.

6.3.2 Docker Environment

The backend image should include the packaged application and a compatible Java runtime. The frontend can be built into static assets and served by a lightweight web server. Multi-stage builds can keep build tools out of runtime images. Images should run as non-root users when supported, avoid unnecessary packages and use explicit version tags. Container logs should be written to standard output and standard error so the deployment platform can collect them.

6.3.3 Kubernetes Environment

The Kubernetes deployment should define Deployments for stateless application components and Services for stable access. Configuration should be externalized through ConfigMaps and Secrets. Resource requests and limits should be based on testing rather than arbitrary guesses. Readiness and liveness probes should reflect actual application health, and startup probes may be appropriate for slow-starting workloads. External access should be routed through a configured Ingress or gateway with TLS.

6.4 Security and Observability

6.4.1 Security Controls

- Use HTTPS for externally accessible traffic.
- Apply authentication and role-based authorization to protected endpoints.
- Keep database credentials, signing keys and tokens out of source code and frontend bundles.
- Use least-privilege database accounts and Kubernetes service accounts.
- Validate all input and use safe database access patterns.
- Keep dependencies and base images updated; scan them in the delivery pipeline.
- Limit public exposure of management endpoints and internal services.
- Define backup, retention and recovery policies for persistent data.
- Record security-relevant events without logging secrets or unnecessary personal data.

6.4.2 Logging, Metrics and Health

Application logs should use consistent structure and include a timestamp, severity, component and correlation identifier where practical. They should capture useful context for diagnosing errors while avoiding sensitive payloads. Health endpoints should distinguish whether the application process is alive from whether it is ready to accept traffic. Metrics can include request count, latency, error rate, JVM memory, thread usage, database connection pool usage and container resource consumption. Monitoring should include alerts for sustained error rates, unavailable replicas, resource exhaustion and database connectivity failures. Distributed tracing may be introduced if requests cross multiple services. Observability is effective only when the team defines how alerts are investigated and who is responsible for responding.

6.5 Implementation Plan

1. Confirm the business domain, users, data model and acceptance criteria.
2. Create the Spring Boot project and implement a small vertical slice from controller to database.
3. Define the API contract and test it independently of the frontend.
4. Create React pages and connect them to the API through a centralized client module.
5. Add authentication and authorization if required by the use case.
6. Write unit, integration and frontend tests; automate the checks in CI.
7. Build Docker images and test them locally with environment-based configuration.
8. Deploy to Kubernetes with probes, Services, resource settings and secret handling.
9. Run functional, failure-recovery and load tests against the target environment.
10. Document release, rollback, backup, monitoring and support procedures.

6.6 Future Enhancements

- Introduce an identity provider and OAuth 2.0 / OpenID Connect for centralized authentication.
- Add role and permission administration with a complete audit trail.
- Adopt automated dependency, container image and configuration security scanning.
- Add a managed secrets solution and automated certificate renewal.
- Introduce caching for selected read-heavy operations after measuring bottlenecks.
- Use event-driven processing for asynchronous tasks were justified by requirements.
- Implement blue-green or canary releases for higher-risk production changes.
- Add dashboards, alerting, distributed tracing and service-level objectives.
- Conduct accessibility reviews and improve responsive behavior across devices.
- Evaluate autoscaling and resilience through measured load and failure-injection tests.

CONCLUSION

This project presents a design for a cloud-native full stack application using React, Spring Boot, Docker and Kubernetes. The architecture separates user interaction, business logic, data persistence and deployment responsibilities. The proposed approach supports repeatable packaging, controlled configuration, health-aware deployment and a foundation for automated delivery. Filtering and optional ratings and reviews demonstrate common user-facing capabilities that can be adapted to the final business domain.

The main lesson is that cloud-native engineering is a combination of application design and operational discipline. Containers and orchestration can improve consistency, but they do not automatically guarantee security, performance or availability. Those qualities must be addressed through validation, authorization, resource planning, automated tests, monitoring, backup and recovery procedures.

REFERENCES

[1] Spring. Spring Boot Reference Documentation. https://docs.spring.io/spring-boot/

[2] React. React Documentation. https://react.dev/

[3] Kubernetes. Kubernetes Documentation. https://kubernetes.io/docs/

[4] Docker. Docker Documentation. https://docs.docker.com/

[5] Fielding, R. T. Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine, 2000.

[6] Newman, S. Building Microservices: Designing Fine-Grained Systems. 2nd edition, O'Reilly Media, 2021.

[7] OWASP Foundation. OWASP Application Security Verification Standard and OWASP Top 10. https://owasp.org/

[8] NIST. Security Strategies for Microservices-based Application Systems. National Institute of Standards and Technology publications and guidance.